

OPTIMIZATION PROJECT

Diana Lorenzo

July 2026

Introduction

The Minimum Enclosing Ball (MEB) problem is a classic task in computational geometry and machine learning, which involves finding the smallest ball that contains a given set of points in $\mathcal{A} = \{a^1, \dots, a^m\} \subset \mathbb{R}^n$. While geometrically intuitive, the MEB problem is essential in modern data science for tasks such as clustering and anomaly detection.

As datasets grow in size and dimensionality, traditional solvers often become impractical. There is a clear need for first-order optimization methods that can provide accurate approximations with lower computational overhead per iteration. The Frank-Wolfe (FW) algorithm, or conditional gradient method, has emerged as an effective projection-free alternative. Unlike projected gradient descent, which updates converge quickly, but they must project back to the constraint set at each iteration in order to maintain a valid solution, the Frank-Wolfe algorithm relies on a Linear Minimization Oracle (LMO).

For the MEB problem, the dual formulation is an optimization problem over the probability simplex Δ_m . This structure is well-suited for Frank-Wolfe methods: the LMO can be implemented efficiently, and the algorithm generates sparse solutions, meaning the ball is defined by only a subset of the original data points. This sparsity is highly beneficial for large-scale data reduction.

This project investigates the performance of the Frank-Wolfe algorithm applied to the MEB problem. We evaluate the convergence behavior of these methods on high-dimensional datasets including MNIST, Fashion-MNIST, and CIFAR-10 and assess their practical utility in identifying anomalies.

1 The Frank-Wolfe Method (FW)

The Frank-Wolfe algorithm, frequently referred to as the conditional gradient method, provides a robust and projection-free framework for solving constrained convex optimization problems of the form:

$$\min_{x \in \mathcal{C}} f(x) \quad (1)$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a convex, continuously differentiable function with an L -Lipschitz continuous gradient, and $\mathcal{C} \subset \mathbb{R}^n$ is a compact and convex set.

1.1 The Linear Minimization Oracle

Given a gradient direction $r = \nabla f(x)$, the LMO solves a linearized subproblem over the feasible domain $\mathcal{C}$:

$$s = \text{LMO}_{\mathcal{C}}(r) \in \arg \min_{s \in \mathcal{C}} \langle s, r \rangle \quad (2)$$

This linear subproblem is often computationally more efficient than a standard Euclidean projection, particularly when $\mathcal{C}$ has specific structural geometries, such as probability simplices and polytopes.

1.2 Algorithmic Scheme

The Frank-Wolfe algorithm refines the solution starting from an initial feasible point $x^{(0)} \in \mathcal{C}$. At each iteration t , the method identifies the Frank-Wolfe atom $s^{(t)}$ via the LMO and updates the current iterate by moving along the feasible descent direction $d^{(t)} = s^{(t)} - x^{(t)}$. The formal procedure is presented in Algorithm 1.

Algorithm 1 Vanilla Frank-Wolfe Algorithm

Require: Initial feasible point $x^{(0)} \in \mathcal{C}$, maximum iterations T , step-size sequence $\{\gamma_t\}_{t=0}^{T-1}$

- 1: **for** $t = 0, 1, \dots, T - 1$ **do**
- 2: **Oracle Call:** $s^{(t)} \leftarrow \arg \min_{s \in \mathcal{C}} \langle s, \nabla f(x^{(t)}) \rangle$
- 3: **Direction Search:** $d^{(t)} \leftarrow s^{(t)} - x^{(t)}$
- 4: **Iterate Update:** $x^{(t+1)} \leftarrow x^{(t)} + \gamma_t d^{(t)}$
- 5: **end for**

1.3 Fundamental Properties

Sparse Approximation Property A defining characteristic of the Frank-Wolfe algorithm is its sparse approximation property. As illustrated in Algorithm 1, the method maintains the iterate $x^{(t)}$ as a convex combination of atoms selected from the feasible domain $\mathcal{C}$. Specifically, after t iterations, the current iterate $x^{(t)}$ can be expressed as a sparse convex combination of at most $t + 1$ atoms discovered throughout the optimization process. This property is particularly advantageous when the feasible domain admits a structure that allows for efficient representation through extreme points or "atoms." The sparse approximation property offers significant practical benefits:

- **Data Reduction:** The algorithm focuses computational effort only on the subset of data points that lie on the boundary of the optimal ball, effectively discarding redundant information.
- **Memory Efficiency:** The sparse representation significantly reduces the memory footprint, ensuring scalability for high-dimensional datasets where explicit projections would be computationally prohibitive.

Sublinear Convergence Analysis For functions with L -Lipschitz continuous gradients, the standard FW method guarantees a convergence rate of $\mathcal{O}(1/t)$ for the duality gap. While theoretically optimal for methods relying solely on linear oracles, this is often slower than the linear convergence rates exhibited by methods using projection or stronger curvature assumptions.

The Zig-zagging problem The slow convergence is particularly evident when the optimal solution resides on the boundary of the feasible domain. In such scenarios, the algorithm starts to zig-zag between the vertices chosen by the LMO to define the face containing x^* . These oscillations cause a sublinear convergence rate.

In the next subsections, we will discuss some variants of the standard Frank-Wolfe, like Pairwise FW and Fully-Corrective FW, that are aimed to overcome this problem and recover linear rates even when x^* lies on the boundary of $\mathcal{C}$.

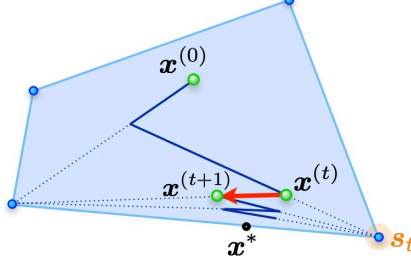


Figure 1: Zig-zagging phenomenon in Frank Wolfe algorithm

2 Frank-Wolfe Variants

To overcome the zig-zagging problem and the resulting sublinear convergence near the boundary, more sophisticated variants of the conditional gradient method have been developed. These approaches structurally modify the update direction to allow for the removal or adjustment of sub-optimal active atoms.

2.1 The Pairwise Frank-Wolfe (PFW)

The Pairwise Frank-Wolfe (PFW) algorithm belongs to the class of Away-step Frank-Wolfe variants. Unlike the vanilla Frank-Wolfe (FW) algorithm, which moves towards a new atom while shrinking the coefficients of all previously active atoms, PFW explicitly exploits the atomic representation of the current iterate to define more effective descent directions. Let the domain $\mathcal{C}$ be a compact convex set represented as the convex hull of a set of atoms $\mathcal{A}$. At iteration t , the current iterate $x^{(t)} \in \mathcal{C}$ is expressed as a convex combination of atoms belonging to the active set $\mathcal{S}^{(t)} \subseteq \mathcal{A}$. We write this expansion as:

$$x^{(t)} = \sum_{v \in \mathcal{S}^{(t)}} \alpha_v^{(t)} v, \quad \text{subject to} \quad \sum_{v \in \mathcal{S}^{(t)}} \alpha_v^{(t)} = 1, \quad \text{and} \quad \alpha_v^{(t)} \geq 0, \quad \forall v \in \mathcal{S}^{(t)}.$$

At each iteration t , the Pairwise Frank-Wolfe (PFW) algorithm identifies two atoms within this atomic decomposition to construct the descent direction:

1. **The “Forward” Atom: ($s^{(t)}$):** This is obtained via the Linear Minimization Oracle (LMO) over the entire atomic set $\mathcal{A}$:

$$s^{(t)} = \arg \min_{s \in \mathcal{A}} \langle s, \nabla f(x^{(t)}) \rangle \quad (3)$$

2. **The “Away” Atom ($v^{(t)}$):** This atom is selected from the current active set $\mathcal{S}^{(t)}$ as the one maximizing the local linear alignment with the gradient, effectively identifying the worst-performing atom in the current convex combination:

$$v^{(t)} = \arg \max_{v \in \mathcal{S}^{(t)}} \langle v, \nabla f(x^{(t)}) \rangle \quad (4)$$

The update direction is defined as $d^{(t)} = d_t^{\text{PFW}} = s^{(t)} - v^{(t)}$. The next iterate is calculated as $x^{(t+1)} = x^{(t)} + \gamma_t d^{(t)}$, where the step size γ_t must be chosen to ensure $x^{(t+1)}$ remains within the feasible region $\mathcal{C}$.

Leveraging the convex combination representation $x^{(t)} = \sum_{v \in \mathcal{S}^{(t)}} \alpha_v^{(t)} v$, the update formula reveals the explicit modification of the weights:

$$x^{(t+1)} = \left(\alpha_{s^{(t)}}^{(t)} + \gamma_t \right) s^{(t)} + \left(\alpha_{v^{(t)}}^{(t)} - \gamma_t \right) v^{(t)} + \sum_{v \in \mathcal{S}^{(t)} \setminus \{s^{(t)}, v^{(t)}\}} \alpha_v^{(t)} v \quad (5)$$

The update rule illustrates that the pairwise step effectively redistributes a mass of exactly γ_t from the away atom $v^{(t)}$ to the forward atom $s^{(t)}$, while keeping the weights of all remaining atoms constant.

The non-negativity constraint on the updated weights, specifically $\alpha_{v^{(t)}}^{(t)} - \gamma_t \geq 0$, restricts the step size to a maximum value defined by the current weight of the away atom: $\gamma_{\max} = \alpha_{v^{(t)}}^{(t)}$.

Within the feasible range $[0, \gamma_{\max}]$, the optimal step size is selected via line search. This approach supports active set maintenance: when γ_t reaches $\gamma_{\max}$, the away atom $v^{(t)}$ is effectively dropped, resulting in a more compact representation of the solution. Pairwise Frank-Wolfe has a weaker theoretical convergence guarantee due to swap steps but often works better in practice, especially for sparse solutions. By actively reducing the influence of ineffective atoms, PFW reduces the oscillations typical of Vanilla FW, and for strongly convex functions, it achieves a linear convergence rate.

Algorithm 2 Pairwise Frank-Wolfe (PFW) Algorithm

Require: Initial iterate $x(0) \in \mathcal{C}$, initial active set $\mathcal{S}(0)$, weights $\alpha(0)$, tolerance $\epsilon > 0$

- 1: **for** $t = 0, 1, \dots, T$ **do**
- 2: $s(t) \leftarrow \arg \min_{s \in \mathcal{A}} \langle \nabla f(x(t)), s \rangle$ ▷ Linear Minimization Oracle
- 3: $v(t) \leftarrow \arg \max_{v \in \mathcal{S}(t)} \langle \nabla f(x(t)), v \rangle$ ▷ Away atom selection
- 4: $d_{\text{PFW}}(t) \leftarrow s(t) - v(t)$ ▷ Pairwise direction
- 5: **if** $\langle -\nabla f(x(t)), d_{\text{PFW}}(t) \rangle \leq \epsilon$ **then**
- 6: **return** $x(t)$ ▷ Stopping criterion
- 7: **end if**
- 8: $\gamma_t \leftarrow \arg \min_{\gamma \in [0, \alpha_{v(t)}(t)]} f(x(t) + \gamma d_{\text{PFW}}(t))$ ▷ Line Search
- 9: $x(t+1) \leftarrow x(t) + \gamma_t d_{\text{PFW}}(t)$
- 10: **Update weights:**
- 11: $\alpha_{v(t)}(t+1) \leftarrow \alpha_{v(t)}(t) - \gamma_t$
- 12: $\alpha_{s(t)}(t+1) \leftarrow \alpha_{s(t)}(t) + \gamma_t$
- 13: **Update active set:**
- 14: $\mathcal{S}(t+1) \leftarrow \{v \in \mathcal{S}(t) \cup \{s(t)\} \mid \alpha_v(t+1) > 0\}$
- 15: **end for**

2.2 Fully-Corrective Frank-Wolfe (FCFW)

While the Pairwise Frank-Wolfe variant relies on a local mass exchange between a single 'away' atom and a 'forward' atom, the Fully-Corrective Frank-Wolfe algorithm performs a global re-optimization of the weights over the entire convex hull of the correction set atoms $\mathcal{A}^{(t)}$. Formally, at each iteration, after appending the new Frank-Wolfe atom $s^{(t)}$ to the correction set, FCFW solves the subproblem:

$$x^{(t+1)} = \arg \min_{x \in \text{conv}(\mathcal{A}^{(t)})} f(x) \quad (6)$$

This approach ensures that the iterate $x^{(t+1)}$ is the true minimizer of the objective function restricted to the current working set, thereby maximizing the progress per outer iteration. FCFW is particularly effective for problems where maintaining a minimal, sparse active set is prioritized, as it prunes redundant atoms that do not contribute to the current optimality.

Algorithm 3 Fully-Corrective Frank-Wolfe (FCFW) Algorithm

Require: Initial iterate $x(0) \in \mathcal{C}$, active set $\mathcal{S}(0)$, tolerance $\epsilon > 0$

```
1: for  $t = 0, 1, \dots, T$  do
2:    $s(t) \leftarrow \arg \min_{s \in \mathcal{A}} \langle \nabla f(x(t)), s \rangle$  ▷ Linear Minimization Oracle
3:   if  $\langle \nabla f(x(t)), x(t) - s(t) \rangle \leq \epsilon$  then ▷ Stopping criterion
4:     return  $x(t)$ 
5:   end if
6:    $\mathcal{A}(t) \leftarrow \mathcal{S}(t) \cup \{s(t)\}$  ▷ Update correction set
7:    $x(t+1) \leftarrow \arg \min_{x \in \text{conv}(\mathcal{A}(t))} f(x)$  ▷ Fully-Corrective step
8:   Update active set:
9:    $\mathcal{S}(t+1) \leftarrow \{v \in \mathcal{A}(t) \mid \alpha_v(t+1) > 0\}$ 
10: end for
```

3 The Minimum Enclosing Ball (MEB)

The Minimum Enclosing Ball (MEB) problem seeks to find the center $\hat{c} \in \mathbb{R}^n$ and radius $\rho \in \mathbb{R}$ of the unique ball of minimal radius that covers a given set of points $\mathcal{A} := \{a^1, \dots, a^m\} \subset \mathbb{R}^n$. The problem can be formulated as a constrained optimization task:

$$\min_{\hat{c} \in \mathbb{R}^n, \rho \in \mathbb{R}} \rho \quad \text{s.t.} \quad \|a^i - \hat{c}\| \leq \rho, \quad \forall i \in 1, \dots, m. \quad (7)$$

By defining $\gamma = \rho^2$, the problem is remodeled as a convex quadratic program with m smooth constraints. For large-scale datasets, where $m \gg n$, the computational overhead of standard projection-based methods (such as Projected Gradient Descent) becomes significant, as the projection onto the intersection of m quadratic constraints is generally expensive. In such contexts, the projection onto an intersection of m quadratic constraints necessitates the solution of an intensive optimization subproblem at every iteration, rendering the approach computationally inefficient.

The Dual Formulation The MEB problem is classically addressed through its Lagrangian dual. By dualizing the constraints, the problem reduces to minimizing a quadratic objective over the unit simplex Δ_m :

$$\min_{u \in \Delta_m} f(x) = x^\top Q x - c^\top x, \quad (8)$$

where $Q = P^\top P$ (with $P \in \mathbb{R}^{n \times m}$ containing the points as columns) and $c_i = \|a^i\|^2$.

This dual formulation is particularly advantageous for the Frank–Wolfe algorithm, as the feasible domain is the standard simplex

$$\Delta_m := \left\{ x \in \mathbb{R}^m : \sum_{i=1}^m x_i = 1, x_i \geq 0, \forall i \in [1 : m] \right\} \quad (9)$$

Motivation for the Frank–Wolfe Approach The transformation of the Minimum Enclosing Ball (MEB) problem into its dual formulation (9) offers decisive computational advantages, establishing it as an ideal use case for the Frank–Wolfe family of algorithms. This suitability is supported by three primary factors:

1. **Simplicity of the Domain:** The constraint is defined by the canonical simplex Δ_m that admits a Linear Minimization Oracle (LMO) with extremely low computational cost as the solution $s = \arg \min_{s \in \Delta_m} \langle s, \nabla f(x) \rangle$ reduces to selecting the index of the minimum component of the gradient.
2. **Efficiency of Line Search:** Since the dual objective $f(x)$ is a quadratic form, the line search to determine the step size γ can be computed in closed form, thereby avoiding costly internal optimization procedures.
3. **Sparsity Property:** It is well established that Frank–Wolfe algorithms generate iterates that are convex combinations of a limited number of atoms from the domain. In the context of the MEB, this results in the construction of so-called *core-sets*: the approximate solution is defined by a restricted subset of points in $\mathcal{A}$ that lie on the boundary of the MEB, thus ensuring a compact and efficient representation of the problem.

Consequently, whereas projection-based methods would require solving a complex optimization subproblem at each iteration, the projection-free approach of Frank–Wolfe allows for effective scaling to large-scale datasets.

3.1 Algorithms implementation

To ensure computational efficiency, particularly when handling large-scale datasets, we implemented the following strategies:

1. **Smart Initialization:** To accelerate convergence and reduce the influence of the initial transient phase, we implemented the geometric dual start (Yildirim, 2008). This involves selecting two points a^α and a^β

from the dataset $\mathcal{A}$ that are farthest from each other. This quantity is approximated by picking a random anchor point a_r , finding the farthest point a^α from it, and subsequently finding the farthest point a^β from a^α .

The algorithm is then initialized by setting the dual weights $x_\alpha = x_\beta = 0.5$, which corresponds to starting with the midpoint of the segment connecting these two points. This strategy effectively provides a high-quality dual start, as it places the initial ball center in a region that is geometrically representative of the dataset.

Vanilla FW case This geometric start prevents the typical long transient phase caused by a random initialization $x(0)$, which forces the algorithm to accumulate suboptimal atoms to correct the initial guess. By setting $x^{(0)} = 0.5a_\alpha + 0.5a_\beta$, the initial linear minimization oracle is immediately evaluated at a central, geometrically representative point $\nabla f(x^{(0)})$. Consequently, the first standard FW update, given by:

$$x^{(1)} = (1 - \gamma_0)x^{(0)} + \gamma_0 s^{(0)}$$

is directed toward a structurally relevant atom $s^{(0)}$ from the very first iteration, significantly accelerating the initial convergence curve.

Pairwise FW case In the case of the Pairwise variant is even more pronounced as it fills the initial active set $\mathcal{S}^{(0)} = \{a_\alpha, a_\beta\}$ with the two most significant boundary atoms, with initial coefficients $\alpha_\alpha^{(0)} = \alpha_\beta^{(0)} = 0.5$. When PFW begins, it can immediately perform a pairwise step:

$$x^{(1)} = x^{(0)} + \gamma_0(s^{(0)} - v^{(0)})$$

where the away atom $v^{(0)} \in \mathcal{S}^{(0)}$ is already a meaningful extreme point, eliminating the need to waste iterations discarding an unrepresentative starting guess.

Crucially, the pairwise squared distances can be computed efficiently directly from the problem data without extra cost: $\|a_i - a_j\|^2 = c_i + c_j - 2Q_{ij}$, where $Q = P^\top P$ and $c_i = \|a_i\|^2$. Therefore, the initial cached product $Qx = 0.5(Q_{:, \alpha} + Q_{:, \beta})$ is obtained in $\mathcal{O}(m)$ operations, avoiding any full matrix-vector product from the very first step.

2. **Gradient Caching:** Given the quadratic form of the dual objective $f(x) = x^\top Qx - c^\top x$, where $Q = P^\top P$, the gradient is expressed as

$\nabla f(x) = Qx - c$. A direct recomputation of the matrix-vector product Qx at each iteration incurs a computational cost of $\mathcal{O}(m^2)$. As the dataset size m grows, this operation quickly becomes a severe computational bottleneck.

To optimize this and make the algorithm scalable, we maintained the product Qx in memory and performed rank-one incremental updates. In Frank-Wolfe variants, the iterate x is modified along sparse directions, for instance, during a Pairwise Frank-Wolfe step, the update takes the form $x_{new} = x_{old} + \gamma(e_{i^*} - e_j)$, where e_{i^*} is the FW vertex, e_j is the away vertex, and γ is the step size. Consequently, the cached matrix-vector product can be updated as follows:

$$Qx_{new} = Qx_{old} + \gamma(Q_{:,i^*} - Q_{:,j}) \quad (10)$$

where $Q_{:,i}$ denotes the i -th column of the matrix Q . This formulation completely bypasses the full matrix multiplication, reducing the per-iteration complexity to $\mathcal{O}(m)$ by only requiring elementary vector operations.

3.2 Exact Line Search and Algorithmic Efficiency

As described in the implementation of our solvers (Standard, Pairwise, and Fully-Corrective Frank-Wolfe), the choice of the step size $\alpha^{(t)}$ at each iteration represents a critical trade-off between numerical accuracy and computational overhead. To ensure optimal performance on large-scale real-world datasets, we integrated an *Exact Line Search* strategy. This approach leverages the specific algebraic structure of the MEB dual problem, proving significantly superior to standard alternatives found in optimization literature:

- **Comparison with Armijo Backtracking:** Armijo requires multiple objective evaluations per iteration to ensure sufficient decrease. Given our dense matrix $Q \in \mathbb{R}^{m \times m}$, this backtracking introduces an unsustainable computational overhead, negating the efficiency of our first-order approach.
- **Comparison with Lipschitz-based Steps ($\frac{1}{L}$):** A fixed step based on $L = \lambda_{\max}(Q)$ relies on the worst-case global curvature over Δ_m . This yields overly conservative, excessively small steps that severely slow down empirical convergence in the early optimization phases.
- **Comparison with Diminishing Step Sizes:** Predefined rules like $\gamma_t = \frac{2}{k+2}$ are computationally free but entirely agnostic to the local

geometry and the LMO descent direction $d^{(t)}$. They force unnecessarily small steps even when a larger, analytically optimal step would yield a much greater objective reduction.

By contrast, the Exact Line Search allows us to determine the globally optimal step size along the search direction in closed form and within $\mathcal{O}(1)$ time complexity, effectively converting a multi-evaluation subproblem into a single direct algebraic formula.

3.2.1 Mathematical Derivation of the Step Size

Recalling the dual formulation of the MEB problem introduced in the previous sections, the objective is to minimize $f(x) = x^\top Qx - c^\top x$ over the probability simplex Δ_m , where $Q = P^\top P$ is a symmetric positive semidefinite matrix, and the gradient is given by $g = \nabla f(x) = 2Qx - c$. Given the current iterate $x^{(t)}$ and a feasible descent direction $d^{(t)}$ (defined uniquely by each FW variant), the updated iterate is expressed as $x^{(t+1)} = x^{(t)} + \alpha d^{(t)}$, where $\alpha \geq 0$ is the step length.

To find the optimal step size, we minimize the one-dimensional restriction of the objective function along the direction $d^{(t)}$, defined as $\phi(\alpha) = f(x^{(t)} + \alpha d^{(t)})$. Substituting the update rule into the objective function yields:

$$\begin{aligned}\phi(\alpha) &= (x^{(t)} + \alpha d^{(t)})^\top Q (x^{(t)} + \alpha d^{(t)}) - c^\top (x^{(t)} + \alpha d^{(t)}) \\ &= x^{(t)\top} Q x^{(t)} + \alpha x^{(t)\top} Q d^{(t)} + \alpha d^{(t)\top} Q x^{(t)} + \alpha^2 d^{(t)\top} Q d^{(t)} - c^\top x^{(t)} - \alpha c^\top d^{(t)}\end{aligned}$$

By exploiting the symmetry of the matrix Q , the cross-product terms satisfy $x^{(t)\top} Q d^{(t)} = d^{(t)\top} Q x^{(t)}$. Grouping the terms by the powers of α , we obtain:

$$\phi(\alpha) = \left(x^{(t)\top} Q x^{(t)} - c^\top x^{(t)} \right) + \alpha \left(2x^{(t)\top} Q d^{(t)} - c^\top d^{(t)} \right) + \alpha^2 d^{(t)\top} Q d^{(t)} \quad (11)$$

The first term in parentheses corresponds exactly to the objective function evaluated at the current iterate, $f(x^{(t)})$. For the linear coefficient in α , we can factor out $d^{(t)}$ and substitute the gradient vector defined as $g = \nabla f(x^{(t)}) = 2Qx^{(t)} - c$:

$$2x^{(t)\top} Q d^{(t)} - c^\top d^{(t)} = (2Qx^{(t)} - c)^\top d^{(t)} = g^\top d^{(t)} \quad (12)$$

Substituting these identifications back into the expression for $\phi(\alpha)$, the one-dimensional optimization subproblem simplifies to a standard convex quadratic parabola:

$$\phi(\alpha) = f(x^{(t)}) + \alpha g^\top d^{(t)} + \alpha^2 d^{(t)\top} Q d^{(t)} \quad (13)$$

To compute the unconstrained optimal step size, we impose the first-order optimality condition by setting the derivative of $\phi(\alpha)$ with respect to α to zero:

$$\frac{d}{d\alpha}\phi(\alpha) = 0 \implies g^\top d^{(t)} + 2\alpha d^{(t)\top} Q d^{(t)} = 0 \quad (14)$$

Solving explicitly for α yields the exact analytical solution for the unconstrained minimizer:

$$\alpha^* = \frac{-g^\top d^{(t)}}{2d^{(t)\top} Q d^{(t)}} \quad (15)$$

Finally, to guarantee that the new iterate remains feasible within the probability simplex Δ_m , this theoretical step size is projected via clipping onto the valid interval $[0, \gamma_{\max}]$, where $\gamma_{\max}$ is determined by the specific Frank-Wolfe variant being executed:

$$\alpha^{(t)} = \max\left(0, \min\left(\gamma_{\max}, \frac{-g^\top d^{(t)}}{2d^{(t)\top} Q d^{(t)}}\right)\right) \quad (16)$$

In our implementation, this calculation is coupled with a *Gradient Caching* strategy. By updating the matrix-vector products incrementally rather than recomputing them at each step, the exact line search is reduced to a highly efficient $\mathcal{O}(1)$ operation.

3.3 Data Pipeline and Primal Recovery

To empirically validate the algorithms on high-dimensional real-world data, we developed a generalized and highly reproducible data loading pipeline. The evaluation is conducted on three standard machine learning benchmarks imported via Keras: MNIST, Fashion-MNIST, and CIFAR-10.

Given that the Minimum Enclosing Ball problem naturally operates in $\mathbb{R}^n$, the multi-dimensional image arrays are flattened into one-dimensional feature vectors and then the pixel values are normalized to the $[0, 1]$ continuous range to ensure numerical stability when computing the inner products. Furthermore each dataset is partitioned into training and testing sets using an 80/20 split, controlled by a globally fixed random seed to guarantee exact reproducibility across all solver runs.

The experimental setup requires training one independent MEB per class (resulting in 10 individual MEBs per dataset). A critical software engineering choice was made regarding the initialization of the quadratic problem. For any given class, the Gram matrix $Q = P^\top P$ (where P is the data matrix) and the norm vector $c_i = \|a_i\|^2$ are computationally expensive to construct. To avoid redundant operations, these structures are built exactly once per class

and passed by reference to the three solvers (Standard, Pairwise, and Fully-Corrective). This ensures that the measurement of time is perfectly unbiased, reflecting solely the algorithmic efficiency of the Frank-Wolfe variants rather than data processing overhead.

Upon convergence of the dual optimization, the primal geometric properties of the minimum enclosing ball are recovered directly from the optimal dual weights $x^* \in \Delta_m$:

- **Center recovery:** Since the optimal center lies in the convex hull of the support vectors, it is obtained via the linear mapping $\hat{c} = Px^*$.
- **Radius recovery:** Instead of calculating distances explicitly, we leverage the theoretical properties of strong duality. At the optimum, the objective value of the dual problem corresponds to the negative squared radius of the primal MEB. Therefore, the radius is analytically derived as $R = \sqrt{-f(x^*)}$, significantly reducing computational complexity at termination.

4 Experimental Results

4.1 Evaluation Metrics and Visualizations

The comparative analysis of the three Frank-Wolfe solvers as we said is conducted across the 3 datasets (totaling 30 independent MEB optimizations). To assess their performance, we focus on the evolution of the Frank-Wolfe Duality Gap, which serves as a rigorous, theoretically upper bound on the primal error.

The results are synthesized through two primary analytical tools:

1. **Comparative Convergence Trajectories:** For each class, we generate overlaid line plots depicting the decay of the Duality Gap. The gap is evaluated against two distinct axes:
 - *Gap vs. Iterations:* This highlights the pure mathematical efficiency and the step-size convergence rate of the algorithms. It visibly demonstrates how Pairwise and Fully-Corrective variants bypass the typical zig-zagging behavior of the Standard FW.
 - *Gap vs. Time:* This axis exposes the actual computational trade-offs. While advanced variants may require fewer iterations to converge, their per-iteration cost (due to active set management and away steps) is higher. Plotting against wall-clock time reveals which method is practically superior.

To properly interpret these trajectories, two specific visualization choices were made, both rooted in optimization theory:

- **Logarithmic y-axis:** Theoretical bounds guarantee that advanced FW variants (such as Pairwise and Fully-Corrective) can achieve global linear convergence when optimizing strongly convex functions over polytopes. This translates to an exponential decay of the suboptimality gap. Plotting the Duality Gap on a logarithmic scale allows us to empirically verify this property, as a linear convergence rate visually manifests as a constant-slope downward line.
- **Moving-Average Smoothing:** Frank-Wolfe methods famously suffer from the "zig-zagging" phenomenon. As the iterate approaches an optimal solution located on a lower-dimensional face of the simplex (rather than at a vertex), the standard algorithm cannot easily drop active vertices and repeatedly bounces between them, causing high-frequency oscillations and local non-monotonicity in the gap. The moving-average filter decouples this local algorithmic noise from the macroscopic asymptotic trend, enabling a fair and clear comparison of the convergence slopes across the three solvers.

2. **Aggregated Summary Tables:** To summarize the macroscopic behavior of the solvers, we compute the average performance metrics (execution time, total iterations required to hit the tolerance threshold, and final reached gap) aggregated over the 10 classes for each method and dataset. This tabular summary provides clear quantitative evidence of the global linear convergence rate achieved by the advanced variants compared to the sublinear rate of the Standard algorithm.

4.2 Stopping Criteria: Duality Gap and Convergence

In convex optimization, determining when an iterative algorithm has sufficiently converged is a critical challenge. Unlike heuristic stopping criteria based on the norm of the step difference ($\|x_k - x_{k-1}\|$) or minor variations in the objective function, the Frank-Wolfe framework provides a rigorous stopping condition known as the Frank-Wolfe Duality Gap.

By exploiting the convexity of the objective function $f(x)$, the linearization at the current iterate x_k yields a valid global lower bound on the minimum. The duality gap $g(x_k)$ is defined as the maximum difference between the current objective value and this linear approximation over the feasible

domain Δ_m :

$$g(x_k) = \max_{s \in \Delta_m} \langle \nabla f(x_k), x_k - s \rangle \quad (17)$$

A profound theoretical property of this gap is that it strictly upper-bounds the true unknown suboptimality quantity: $f(x_k) - f(x^*) \leq g(x_k)$. Thus, driving the FW gap to zero rigorously guarantees convergence to the global optimum.

From an implementation perspective, calculating $g(x_k)$ incurs zero additional computational cost. The point s that maximizes the inner product is precisely the extreme vertex e_{i^*} identified by the Linear Minimization Oracle (LMO) during the search direction computation. Therefore, the gap is simply evaluated at each iteration as:

$$g(x_k) = \langle \nabla f(x_k), x_k \rangle - \nabla f(x_k)_{i^*} \quad (18)$$

Based on this theoretical foundation, the execution of our three solvers (Standard, Pairwise, and Fully-Corrective) is governed by two complementary stopping criteria:

- **Tolerance Threshold (ϵ):** The primary condition for declaring successful convergence is $g(x_k) \leq \epsilon$. In our experimental setup, we defined a strict tolerance threshold of $\epsilon = 10^{-10}$ to ensure a high-precision geometric approximation of the Minimum Enclosing Ball.
- **Maximum Iterations (T_{max}):** To guarantee finite termination in scenarios where extreme numerical precision limits (e.g., floating-point drift) prevent the gap from perfectly crossing the ϵ threshold, a hard cutoff is enforced at $T_{max} = 10^7$ iterations.

4.3 Performance Comparison Across Datasets

As previously anticipated, our implementation compared the three algorithms on 3 distinct datasets of 10 classes each, building a single ball for each one, for a total of thirty independent Minimum Enclosing Balls.

Every run shared an identical time budget of 15 seconds per class and a duality-gap tolerance of $\epsilon = 10^{-10}$, so that the numbers reported below are comparable and reflect the intrinsic efficiency of each algorithm on the common quadratic problem.

As expected, the per-class breakdown of the convergence time in the heatmap above shows a remarkably consistent pattern across classes. Each cell reports the time to convergence for one (class, solver) pair, colored on a common green-to-red scale shared across all three datasets.

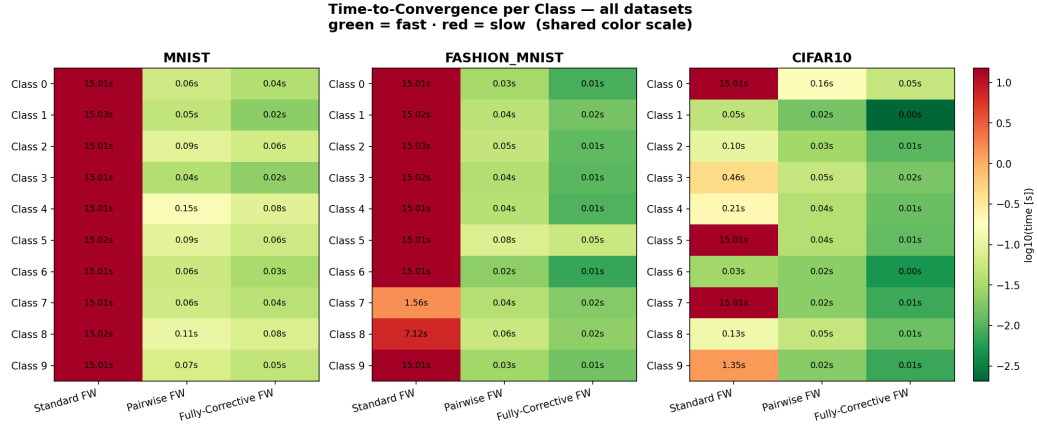


Figure 2: All Classes and Datasets

The Standard-FW column is uniformly red on MNIST and Fashion MNIST (at 15 s budget on almost every class except for 2 classes), while the Pairwise and Fully-Corrective columns stay uniformly green, with small variation from class to class.

This effect is not uniform across CIFAR-10, however, since core-set size depends on class geometry. Compact object classes (e.g., vehicles) yield minimal core-sets, while high-variability classes (e.g., animals with diverse poses) remain as geometrically complex as MNIST, causing Standard FW to time out there as well. This peculiarity will be further discussed in next sections.

To conclude, it is clear that, even if convergence is reached also by the simpler Standard Frank Wolfe strategy in 2 classes of Fashion MNIST and 7 of CIFAR, the corresponding Active sets methods are outperforming even in those cases.

Given this homogeneity, averaging over classes is a solid choice, which will make the cross-dataset, cross-solver comparison easier to analyze.

Table 1: Performance comparison of Frank-Wolfe variants across different datasets (calculated only on converged classes for the Standard FW method).

Dataset	Solver	Converged	Avg Time (s)	Avg Iter.	Avg Final Gap
MNIST	Standard FW	0/10	N/A	N/A	N/A
	Pairwise FW	10/10	0.08	564	9.15×10^{-11}
	Fully-Corrective FW	10/10	0.05	25	8.25×10^{-12}
Fashion-MNIST	Standard FW	2/10	4.34	71100	9.97×10^{-11}
	Pairwise FW	10/10	0.04	302	8.91×10^{-11}
	Fully-Corrective FW	10/10	0.02	14	1.89×10^{-11}
CIFAR-10	Standard FW	7/10	0.33	6393	9.37×10^{-11}
	Pairwise FW	10/10	0.04	544	7.80×10^{-11}
	Fully-Corrective FW	10/10	0.01	19	1.36×10^{-11}

The difference is clear: on MNIST, the Standard Frank-Wolfe method fails to converge within the 15-second time budget on every single class (0/10 instances converged). On Fashion-MNIST and CIFAR-10, it manages to converge on only 2 and 7 classes out of 10, respectively. For these few converged instances, while the average final gap reaches the target threshold of around 10^{-11} , the computational cost is heavily asymmetric: on Fashion-MNIST, the converged classes require an average of 71100 iterations and 4.34 seconds, whereas on CIFAR-10, convergence is reached much faster (0.33 seconds on average and 6393 iterations). In contrast, the Pairwise and Fully-Corrective variants achieve robust convergence across all 10 classes on every dataset (10/10). They reach optimality in a fraction of a second (between 0.01 and 0.08 seconds) and drastically reduce both the iteration count—requiring as few as 14 to 19 iterations for the Fully-Corrective variant on Fashion-MNIST and CIFAR-10—and the final duality gap down to the order of 10^{-11} and 10^{-12} (with Fully-Corrective reaching 1.36×10^{-11} on CIFAR-10). It is important to underline that the Standard method behaves exactly as the theory predicts for a first-order conditional gradient when the optimum lies on the boundary of the feasible simplex (Lacoste-Julien and Jaggi, 2015). The severe drop in convergence and the systematic timeouts are a clear sign of the zig-zagging phenomenon introduced in Section 1.3.

4.3.1 Iteration Complexity and the Cost of Zig-Zagging

To understand the algorithmic efficiency of each method, we can look at the decay of the duality gap as a function of the iteration count.

On such a graph, the sublinear $\mathcal{O}(1/t)$ rate appears as a curve that flattens

more and more as it descends, whereas a linear (geometric) rate appears as a straight downward line of constant slope. Notice that due to the log-log scale (used here to accommodate the wide range of iteration counts across methods) the flattening is compressed and less visually apparent than on the time based plot on the right, where the standard linear time axis makes the effect unambiguous.

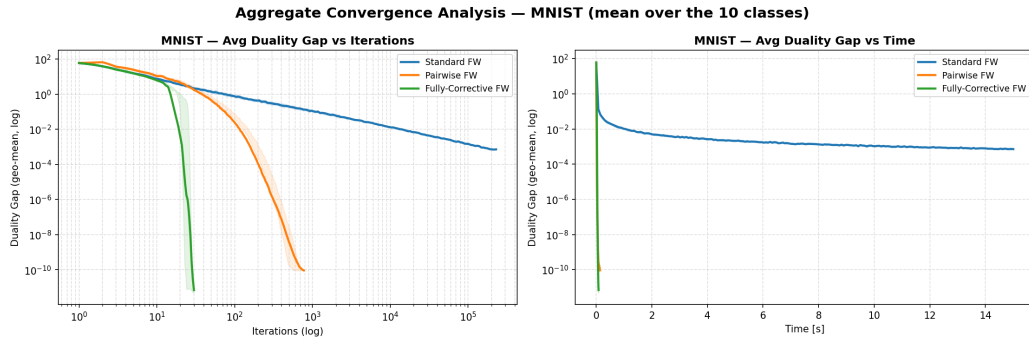


Figure 3

In the graph above we can analyze the geometric mean of the gap on MNIST dataset, with respect to the iterations (and time) by each algorithm.

The shaded band spans the 25–75% inter-class range: the fact that this band stays pretty narrow is a proof of the fact that the behaviour we describe is shared by every class.

Looking at the iterations graph on the left, we can see how the Standard method descends quickly during the first handful of iterations and then settles into an almost flat line: after the initial transient it keeps oscillating between the few vertices that define the optimal face of the simplex, making only marginal progress per step.

This is exactly the behavior anticipated by the convergence theory: the duality gap is upper bounded by $\mathcal{O}(C_f/t)$ through the curvature constant C_f of the objective function (Theorem 2: Jaggi, 2013), and lower bounded by $\Omega(1/t^{1+\delta})$ (as formalized by Canon and Cullum and revisited in Lacoste-Julien and Jaggi, 2015) whenever the optimal solution lies on a face of the polytope. Since this is precisely our case, where a convex quadratic objective is minimized over the simplex yielding a sparse optimal solution, the true convergence rate is therefore trapped between these two bounds. In opposition, both active-set variants exhibit a constant-slope decay that is the visual proof of a linear convergence rate: once they have identified the correct support, they converge after a few hundred (Pairwise) or a few dozen (Fully-Corrective) iterations.

The reason lies in the update direction itself: the FW standard step, moving only along $d_t^{\text{FW}} = s^{(t)} - x^{(t)}$, can never explicitly discard a bad atom, but it can only shrink it slowly. This is what forces the iterate to zig-zag across the optimal face. The Pairwise step breaks this logic, because a single move along $d_t^{\text{PFW}} = s_t - v_t$ grows the "right" vertex and at the same time starves the "wrong" one. The Fully-Corrective step instead goes further, resolving the problem exactly over the active set and zeroing out every residual sub-optimality, thanks to the fact that it uses the pairwise under the hood.

4.3.2 Time Efficiency and Per-Iteration Trade-offs

Counting iterations tells only a part of the story, because the three methods do not pay the same price per step.

The Fully-Corrective variant is by far the most economical in terms of iteration count (19 on average for CIFAR-10), against the several hundred of Pairwise FW (544). However, each of its iterations is considerably heavier: it appends the new atom to the active set and then re-optimizes all active weights through an inner loop restricted to the small active sub-matrix. The only remaining global operation is the exact refresh of the cached product Qx , which is computed as $Q_{:,A}x_A$ over the active columns only, costing $O(m|A|)$ per outer step instead of a full $O(m^2)$ matrix-vector product. The Pairwise method, instead, performs only a single weight swap per iteration, which the gradient caching scheme reduces to $O(m)$. The net effect on time is, of course, problem-dependent. But analyzing Table 1 and the heatmap, we can see how in our three examples the Fully-Corrective is now the fastest method on average on every dataset: its far smaller iteration count more than compensates for the heavier outer step, once the latter is restricted to the active columns.

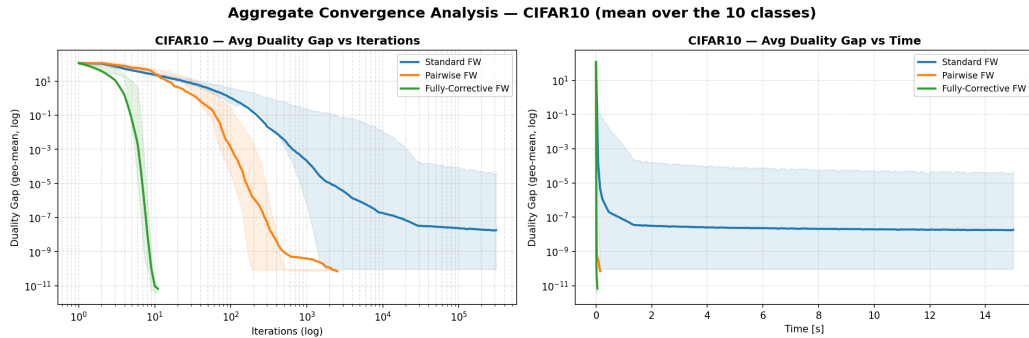


Figure 4

In Figure 3 (right graph) we can see how for MNIST, where the point

clouds are dense and each correction touches a larger active set, the two variants stay close in time (0.08 s for Pairwise against 0.05 s for Fully-Corrective), even though Pairwise needs roughly eleven times more iterations. On the other hand, on the lower cardinality CIFAR, the Fully-Corrective pulls slightly ahead (0.01 s versus 0.04 s), since its minimal core-sets keep both the active set and the outer iteration count (19 on average) extremely small. This is proven in Figure 4 (right graph). The practical lesson is that the correction step of Fully-Corrective, once its global cost is confined to the active columns, translates into the shortest convergence time: the margin over Pairwise is present even on the denser MNIST classes and on CIFAR-10. In any case, both variants dramatically outperform the vanilla method.

4.3.3 Dataset-Dependent Behaviour of the Standard Method

As previously anticipated, the Standard method exhibits an uneven behavior across datasets. On MNIST it times out on all ten classes. On Fashion-MNIST, the classes that actually reach convergence do so in an average time of 4.34 s (with 2 out of 10 classes converging within the limit). On CIFAR-10, the method shows a much higher success rate, converging on 7 out of 10 classes with a remarkably low average time of 0.33 s for the converged runs. For several CIFAR classes, in fact, the standard FW algorithm converged within a fraction of a second (this difference is also proved by the different widths of the shaded bands in Figure 4).

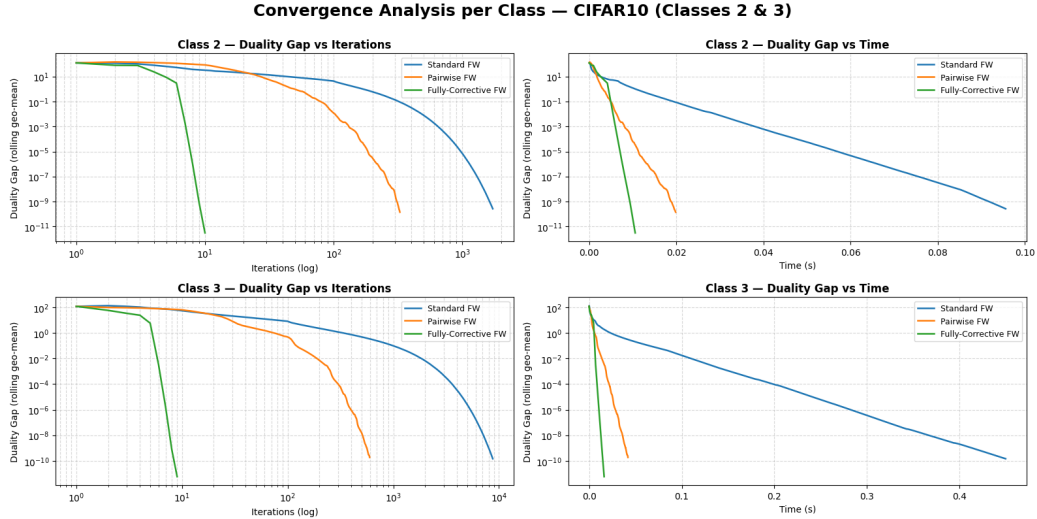


Figure 5

Figure 5 therefore shows classes 2 and 3 separately and perfectly testifies

to what we have just mentioned. While both classes eventually reach convergence, Class 2 enters its linear phase almost immediately, reaching the target duality gap in less than 0.1 seconds and under 2000 iterations. Conversely, Class 3 exhibits a noticeably slower convergence path, requiring around 0.45 seconds and nearly 10000 iterations to minimize the gap. The $O(1/k)$ upper bound for Standard FW describes, in fact, the worst case, which happens while the iterate is still oscillating near the optimal face. However, when the vanilla method accumulates just the atoms that support the solutions and no superfluous ones, the method enters a linear trend. Whether this occurs before the time limit depends heavily on the geometry of the class. It happens, in fact, for several CIFAR-10 categories, whose optimal support is comparatively small and cleanly separated, and essentially never for the denser MNIST digit clouds. Instead, the active set variants (Away-steps and Pairwise FW) and the Fully-Corrective FW directly discard the wrong atoms rather than waiting for them to be excluded. For this reason, they exhibit superior performance, achieving a 10/10 convergence rate across all datasets with average times consistently below 0.1 seconds (reaching 0.04 s for Pairwise and 0.02 s for Fully-Corrective on Fashion-MNIST) and minimizing the final gap down to magnitudes of 10^{-11} and 10^{-12} .

5 Anomaly Detection

The Ball constructed in the previous iterations can be seen as compact models of "what a normal member of that class looks like," and a new sample can be scored by how far it is from that model.

5.1 The Geometric Anomaly Score

For a test point z belonging (by its true label) to a class whose MEB has center $\hat{c}$ and radius R , the anomaly score is the distance from the center, normalized by the radius of the class ball:

$$\text{ratio}(z) = \frac{\|z - \hat{c}\|}{R}.$$

Recall from Section 3 that $\hat{c} = Px^*$ and $R = \sqrt{-f(x^*)}$ are recovered via strong duality without ever computing a distance explicitly. The normalization by R makes the score comparable across classes of different scale, with an immediate interpretation:

- $\text{ratio}(z) > 1$: z lies strictly outside the class ball, so it's an outlier
- $\text{ratio}(z) \approx 1$: z sits on the boundary, so it's still a plausible member

- $\text{ratio}(z) \ll 1$: z it's in the interior of the ball, so a standard example

This is a direct consequence of the MEB constraint $\|a_i - \hat{c}\|^2 \leq R^2$, which every training point satisfies by construction. A test point is anomalous exactly when it violates the constraint that every training example is satisfying.

Expanding $\|z - \hat{c}\|^2 = \|z\|^2 - 2z^\top \hat{c} + \|\hat{c}\|^2$ lets the whole test set be scored with a single matrix-vector product per class. Notice that, because the true label is known and used to select which ball to score against, this is a supervised check rather than an open one testing whether a point belongs to any known class.

5.2 Results and Interpretation

Table 2. Number of Outliers for each Dataset

Dataset	Test samples	Samples outside the MEB (ratio > 1)	Fraction
MNIST	14000	67	$\approx 0.48\%$
Fashion-MNIST	14000	31	$\approx 0.22\%$
CIFAR-10	12000	22	$\approx 0.18\%$

Table 2 summarizes the counts of all the test samples falling outside of their class ball. The percentages are very small, which is exactly what one would expect. To understand what could have actually affected these numbers, we inspected the test points by their ratio and displayed the most and least anomalous one (highest or lowest ratio).

On MNIST the pattern is exactly what a human would expect (Figure 6). The most anomalous digits. For instance the class 7 sample scoring ratio = 1.15, the highest in the whole dataset, is drawn in a very atypical way in those far instances. The least anomalous digits, clustering around ratio ≈ 0.81 – 0.85 , are instead clean and upright. We are thus detecting genuine handwriting "errors" rather than arbitrary noise.

Fashion-MNIST (Figure 7) tells a similar but slightly different story: its most anomalous items (ratio up to about 1.07) are items photographed in unusual poses or with atypical dress shapes, while the least anomalous ones fall much lower, around ratio ≈ 0.42 – 0.45 . The gap between the two extremes is wider than on MNIST for a specific reason: a handful of distinctive pieces stretch R outward while the majority of ordinary items remains fixed so the same typical item now reads as more normal simply because the normalized R got longer.

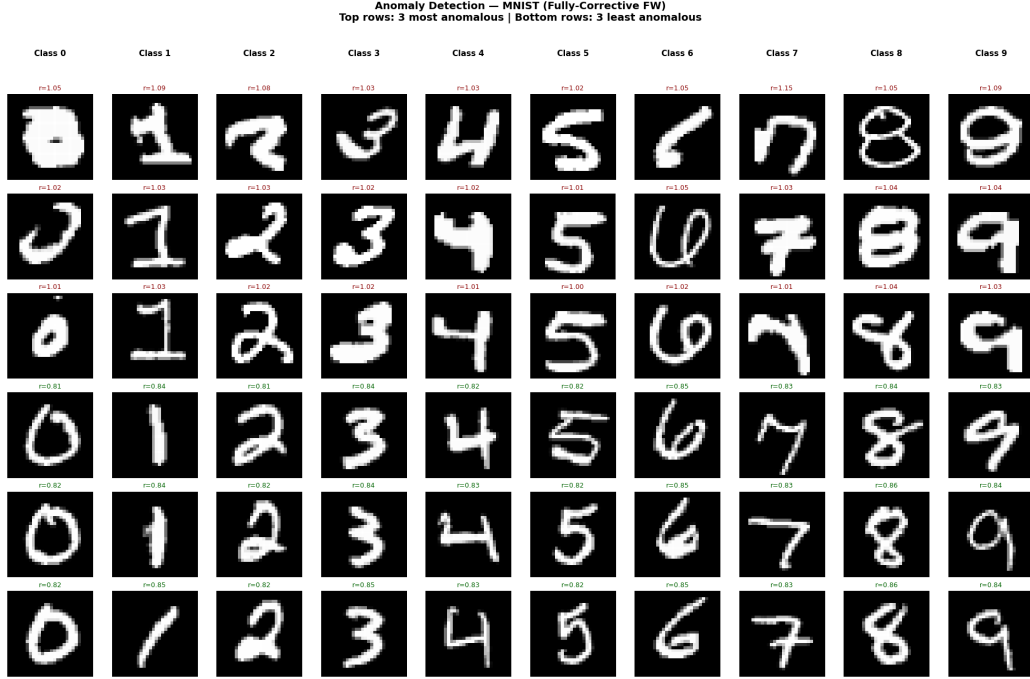


Figure 6

CIFAR-10 pushes this effect to the extreme (Figure 8). Its least anomalous images drops to ratio ≈ 0.18 – 0.25 , since here we have centred and clean photographs of the objects. The most anomalous digits rise to ratio = 1.15. This is an example of the ball’s min-max nature: R is set by the single farthest point, not by any average, so one messy or weirdly colored image inflates the radius for the whole class and drags every ordinary sample down to a very small ratio. The MEB is thus a sensitive model: sharply reactive to a genuine outlier, which makes it a crisp detector, but also one where a single extreme point can reshape the scale for everyone else.

5.3 Algorithms Agreement on Anomalies

It is important to denote that the three versions of the FW algorithm produce identical rankings and ratios on every class and every dataset. This is not a coincidence but a consequence of the convexity of the dual problem: the Minimum Enclosing Ball is unique, so any method that solves the same quadratic program converges to the same center and radius, and thus anomalies score.

What is genuinely interesting is that even the Standard method, which in most cases was stopped by the time limit with a residual gap way higher



Figure 7

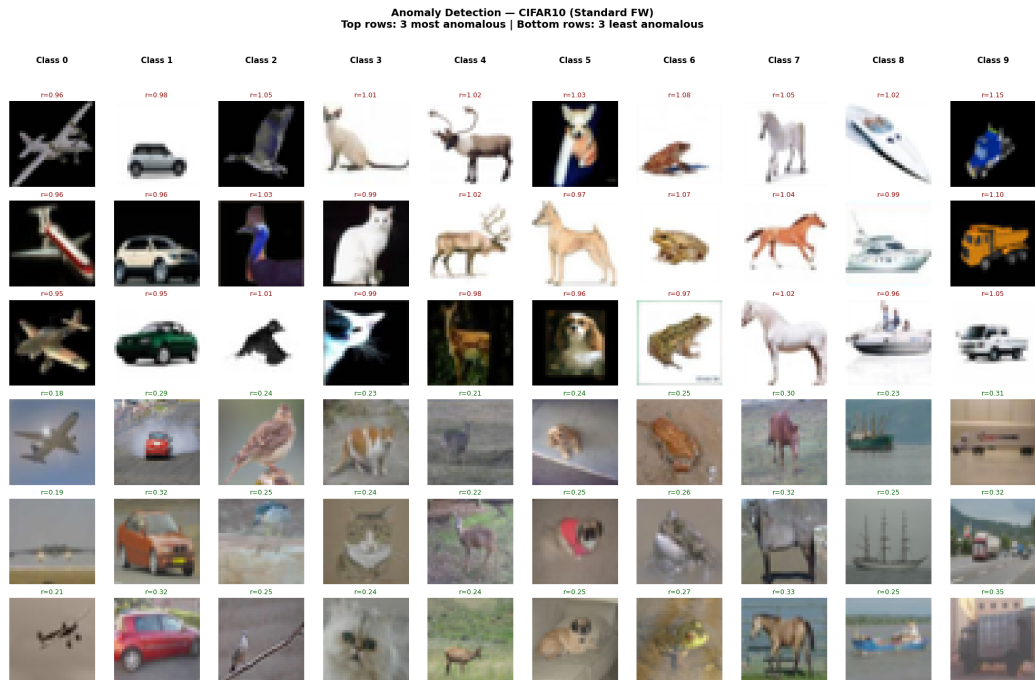


Figure 8

than the active set variants, still yields the same ratios. The reason is that the center and radius stabilize long before the duality gap reaches the final precision required. In conclusion, we can see how sometimes, in real world applications even a theoretically worst solution can lead to the same results of more complete and sophisticated ones.

6 LLM Usage Disclosure Statement

For the completion of this project, we primarily utilized **Gemini** as a supportive tool for the following tasks:

6.1 Problem Understanding and Theoretical Analysis

At the beginning we used the LLM to deeply understand the concepts presented in the papers, in order to facilitate the connection with what we have discussed in class and what we should have done for our project.

6.2 Algorithm Design and Coding

While the core algorithmic logic and theoretical adaptations were developed independently, we utilized a Large Language Model (LLM) as an advanced debugging and optimization assistant.

Among the part where LLM helped us while debugging our code, we highlight for sure the following ones:

- Assisting in the efficient implementation of memory-caching strategies.
- Advanced NumPy vectorization and numerical stability handling (e.g., active set sub-matrix extraction for the FCFW inner iterations, and machine precision thresholds).
- Writing the theoretical MEB initialization into our code
- Importing the datasets correctly and solving issues with Cifar server being very slow

Finally, the LLM was used to automate the generation of graphs, and helped us write in Latex faster (mathematical formulas, tables,etc) or correct typos.